

Interactive and Automatic Theorem Proving in the First Order Theory of Combinators using Agda

Ana Bove¹, Peter Dybjer¹, Andrés Sicard-Ramírez²

¹ Chalmers tekniska högskola, Göteborg, Sweden

² EAFIT Medellín, Colombia

AIM XIII

Göteborg, 8 April, 2011

Combining three strands of research

- Aczel's *first order theory of combinators* (and related systems) as logics for *general recursive* functional programs. Dybjer (1985), Dybjer and Sander (1989).
- Proving properties of Haskell programs by translating them into *first order logic* and using *automatic theorem provers*. Claessen and Hamon "The Cover Translator" (2003).
- Using automatic theorem provers for first order logic for proving theorems in dependent type theory. Tammet and Smith (1996) "Gandalf", Abel, Coquand, Norell (2005) "AgdaLight". (Cf also talks by Foster and Kanso at this meeting.)

A First Order Theory of Combinators (FOTC)

Aczel (1974): “The interpretation of Martin-Löf type theory in a first order theory of combinators”.

$$\begin{aligned} t &::= x \mid t \, t \mid K \mid S \\ \Phi &::= \perp \mid \top \mid \Phi \wedge \Phi \mid \Phi \vee \Phi \mid \neg \Phi \mid \forall x. \Phi \mid \exists x. \Phi \mid t = t \mid \\ &\quad \mathcal{N}(t) \mid \mathcal{P}(t) \mid \mathcal{I}(t) \end{aligned}$$

Cf *The Logical Theory of Constructions (LTC)*, *Frege Structures*, etc.

A Logic for PCF with totality predicates

$$\begin{aligned} t &::= x \mid t \ t \mid \lambda x. t \mid \text{true} \mid \text{false} \mid \text{if} \mid 0 \mid \text{succ} \mid \text{pred} \mid \text{iszero} \mid \text{fix} \\ \Phi &::= \perp \mid \top \mid \Phi \wedge \Phi \mid \Phi \vee \Phi \mid \neg \Phi \mid \forall x. \Phi \mid \exists x. \Phi \mid t = t \mid \\ &\quad \text{Bool}(t) \mid \mathcal{N}(t) \end{aligned}$$

We may want to add other inductively defined predicates.

A first order theory of combinators for PCF with totality predicates

$$\begin{aligned}
 t &::= x \mid t \ t \mid \text{true} \mid \text{false} \mid \text{if} \mid 0 \mid \text{succ} \mid \text{pred} \mid \text{iszero} \mid f \\
 \Phi &::= \perp \mid \top \mid \Phi \wedge \Phi \mid \Phi \vee \Phi \mid \neg \Phi \mid \forall x. \Phi \mid \exists x. \Phi \mid t = t \mid \\
 &\quad \text{Bool}(t) \mid \mathcal{N}(t)
 \end{aligned}$$

where f ranges over new combinators defined by (recursive) equations

$$f \ x_1 \ \cdots \ x_n = e[f, x_1, \dots, x_n]$$

Pattern matching can be implemented by `if`, `pred`, `iszero`.

Agda as a proof assistant for first order theories

Agda as A Logical Framework (ALF)! Syntax and inference rules of the logic are implemented by constants in a lambda calculus with dependent types (Martin-Löf's Logical Framework 1986, Edinburgh Logical Framework 1986). For example,

```
postulate
```

```
  _∨_ : Set → Set → Set
```

```
  inl : {A B : Set} → A → A ∨ B
```

```
  inr : {A B : Set} → B → A ∨ B
```

```
  case : {A B C : Set} → (A → C) → (B → C) → A ∨ B → C
```

Note that these are *axiom schemata* in first order logic.

Using data instead of postulates

If we use Agda's data declaration

```
data _∨_ (A B : Set) : Set where
  inl : A → A ∨ B
  inr : B → A ∨ B
```

we can write proofs using $\vee$ -elimination using pattern matching:

```
commOr : {A B : Set} → A ∨ B → B ∨ A
commOr (inl a) = inr a
commOr (inr b) = inl b
```

rather than

```
commOr c = case inr inl c
```

We will use data for all logical quantifiers, equality, and inductively defined predicates, such as the totality predicates, in FOTC.

Quantifiers

The domain of individuals:

```
postulate  
  D : Set
```

The existential quantifier:

```
data  $\exists$  (P : D  $\rightarrow$  Set) : Set where  
  _,_ : (x : D)  $\rightarrow$  P x  $\rightarrow$   $\exists$  P
```

The universal quantifier is implemented by Agda's dependent function types.

Calling an automatic theorem prover

If we want to prove a theorem automatically, we

- 1 postulate it in Agda;
- 2 add a *pragma* `{-# ATP prove ... #-}` which instructs the program `agda2atp` (written by Andrés) to translate it to TPTP and then ask the automatic theorem provers (E, Equinox, Vampire, Metis) to try to prove it.

```
postulate
  commOr : {A B : Set} → A ∨ B → B ∨ A
{-# ATP prove commOr #-}
```

The law of the excluded middle

The ATPs use classical logic:

postulate

```
  lem : {A : Set} → A ∨ ¬ A  
{-# ATP prove lem #-}
```

There are no schemata in TPTP

The Agda type

$$\{A\ B : \text{Set}\} \rightarrow A \vee B \rightarrow B \vee A$$

corresponds to the schema

$$\Phi \vee \Psi \supset \Psi \vee \Phi$$

where Φ and Ψ range over arbitrary well-formed formulae in a first order theory. But this cannot be expressed in TPTP. Instead Andrés translates the type into the TPTP formula

$$P \vee Q \rightarrow P \vee Q$$

where P and Q are new 0-ary predicate symbols.

Agda as an interface to classical first order theorem provers

We shall look at a theorem in the first order theory due to Stanovsky (2008) with one binary predicate symbol and two axioms (left and right distributivity).

- No ATP can prove the theorem in one step.
- Andrés has formalized Stanovsky's proof. It gives rise to approximately 200 lines of Agda code.
- We show a combined interactive and automatic proof using (a variant of) the equational reasoning module from Agda's standard library!

Stanovsky's first order theory in Agda

```

postulate
  _·_ : D → D → D

  leftDistributive  : ∀ x y z → x · (y · z) ≡ (x · y) · (x · z)
  rightDistributive : ∀ x y z → (x · y) · z ≡ (x · z) · (y · z)
{-# ATP axiom leftDistributive #-}
{-# ATP axiom rightDistributive #-}

```

A theorem which cannot be proved automatically:

```

postulate
  stanovsky : ∀ u x y z →
    (x · y · (z · u)) · (( x · y · ( z · u)) · (x · z · (y · u))) ≡
    x · z · (y · u)
{-# ATP prove stanovsky #-}

```

```
-- Equinox 5.0alpha (2010-06-29): TIMEOUT (180 seconds)
```

A combined interactive and automatic proof

```
stanovsky u x y z =
```

```
begin
```

```
  xy·zu · (xy·zu · xz·yu) ≡⟨ j1-5 ⟩
```

```
  xy·zu · (xz · xu·yu · (y·zu · xz·yu)) ≡⟨ j5-9 ⟩
```

```
  xy·zu · (xz · xyu · (yxz · yu)) ≡⟨ j9-14 ⟩
```

```
  xz · xyu · (yz · xyu) · (xz · xyu · (y·xu · z·yu)) ≡⟨ j14-20 ⟩
```

```
  xz · xyu · (y·xu · (y·yu · z·yu) · (z · xu·yu · (y·xu · z·yu))) ≡⟨ j20-23 ⟩
```

```
  xz · xyu · (y · xu·zu · (z · xu·yu · (y·xu · z·yu))) ≡⟨ j23-25 ⟩
```

```
  (xz · xyu) · (y · xu·zu · (z·xu · y·xu · z·yu)) ≡⟨ j25-30 ⟩
```

```
  xz · xyu · (y·zy · xzu) ≡⟨ j30-35 ⟩
```

```
  xz·yu
```

■

```
postulate
```

```
  j1-5 : xy·zu · (xy·zu · xz·yu) ≡ xy·zu · (xz · xu·yu · (y·zu · xz·yu))
```

```
{-# ATP prove j1-5 #-}
```

```
...
```

Abbreviations: $ab \equiv a \cdot b$ and $abc \equiv a \cdot b \cdot c$

PCF: basic combinators

```
postulate
  _·_      : D → D → D
  true     : D
  false    : D
  if_then_else_ : D → D → D → D
  zero     : D
  succ     : D → D
  pred     : D → D
  isZero   : D → D
  error    : D
```

Actually, `if`, `succ`, `pred`, `isZero` : `D`, but then the above versions are definable, and easier to use for the theorem prover (and the human).

PCF: conversion rules

```
postulate
```

```
  if-true  :  $\forall d_1 \{d_2\} \rightarrow \text{if true then } d_1 \text{ else } d_2 \equiv d_1$ 
```

```
  if-false :  $\forall \{d_1\} d_2 \rightarrow \text{if false then } d_1 \text{ else } d_2 \equiv d_2$ 
```

```
{-# ATP axiom if-true #-}
```

```
{-# ATP axiom if-false #-}
```

```
postulate
```

```
  pred-0 : pred zero  $\equiv$  zero
```

```
  pred-S :  $\forall d \rightarrow \text{pred (succ d)} \equiv d$ 
```

```
{-# ATP axiom pred-0 #-}
```

```
{-# ATP axiom pred-S #-}
```

```
postulate
```

```
  isZero-0 : isZero zero  $\equiv$  true
```

```
  isZero-S :  $\forall d \rightarrow \text{isZero (succ d)} \equiv \text{false}$ 
```

```
{-# ATP axiom isZero-0 #-}
```

```
{-# ATP axiom isZero-S #-}
```


PCF: discrimination rules

```
postulate
  true≠false : ¬ (true ≡ false)
  0≠S        : ∀ {d} → ¬ (zero ≡ succ d)
{-# ATP axiom true≠false #-}
{-# ATP axiom 0≠S #-}
```

PCF: total booleans

```
data Bool : D → Set where
  tB : Bool true
  fB : Bool false
{-# ATP axiom tB #-}
{-# ATP axiom fB #-}

caseBool : (P : D → Set) → P true → P false →  $\forall \{b\} \rightarrow$ 
           Bool b → P b
caseBool P Pt Pf tB = Pt
caseBool P Pt Pf fB = Pf
```

Note that `caseBool` is a *schema*, so TPTP doesn't understand it.

PCF: total natural numbers

```

data N : D → Set where
  zN : N zero
  sN : ∀ {n} → N n → N (succ n)
{-# ATP axiom zN #-}
{-# ATP axiom sN #-}

indN : (P : D → Set) →
  P zero →
  (∀ {n} → N n → P n → P (succ n)) →
  ∀ {n} → N n → P n
indN P P0 h zN      = P0
indN P P0 h (sN Nn) = h Nn (indN P P0 h Nn)

```

Note that `indN` is a *schema*, so TPTP doesn't understand it.

Adding a combinator for addition

```
postulate
  _+_  : D → D → D

  +-0x : ∀ d → zero  + e ≡ e
  +-Sx : ∀ d e → succ d + e ≡ succ (d + e)
{-# ATP axiom +-0x #-}
{-# ATP axiom +-Sx #-}
```

Note that `+` can be defined by one recursive equations using `if_then_else_`, `pred` and `isZero`.

Addition is a total function

Do the case analysis manually:

```
+ -N : ∀ {m n} → N m → N n → N (m + n)
```

```
+ -N {n = n} zN _ = prf
```

```
  where
```

```
    postulate prf : N (zero + n)
```

```
    {-# ATP prove prf #-}
```

```
+ -N {n = n} (sN {m} Nm) Nn = prf (+ -N Nm Nn)
```

```
  where
```

```
    postulate prf : N (m + n) → N (succ m + n)
```

```
    {-# ATP prove prf #-}
```

Addition is a total function - version 2

Instantiate the induction rule:

```
indN-instance : (x : D) →  
  N (zero + x) →  
  (∀ {n} → N n → N (n + x) → N (succ n + x)) →  
  ∀ {n} → N n → N (n + x)  
indN-instance x = indN (λ i → N (i + x))  
  
postulate  
  +-N1 : ∀ {m n} → N m → N n → N (m + n)  
{-# ATP prove +-N1 indN-instance #-}
```

Commutativity of addition

```
+-comm :  $\forall \{m\ n\} \rightarrow N\ m \rightarrow N\ n \rightarrow m + n \equiv n + m$ 
```

```
+-comm {n = n} zN _ = prf
```

```
  where
```

```
    postulate prf :  $zero + n \equiv n + zero$ 
```

```
    {-# ATP prove prf +-rightIdentity #-}
```

```
+-comm {n = n} (sN {m} Nm) Nn = prf (+-comm Nm Nn)
```

```
  where
```

```
    postulate prf :  $m + n \equiv n + m \rightarrow succ\ m + n \equiv n + succ\ m$ 
```

```
    {-# ATP prove prf x+S y≡S[x+y] #-}
```

A partial function: greatest common divisor

postulate

gcd : D → D → D

gcd-00 : gcd zero zero ≡ error

gcd-S0 : ∀ n → gcd (succ n) zero ≡ succ n

gcd-0S : ∀ n → gcd zero (succ n) ≡ succ n

gcd-S>S : ∀ m n → GT (succ m) (succ n) →
gcd (succ m) (succ n) ≡ gcd (succ m - succ n) (succ n)

gcd-S≤S : ∀ m n → LE (succ m) (succ n) →
gcd (succ m) (succ n) ≡ gcd (succ m) (succ n - succ m)

{-# ATP axiom gcd-00 #-}

{-# ATP axiom gcd-S0 #-}

{-# ATP axiom gcd-0S #-}

{-# ATP axiom gcd-S>S #-}

{-# ATP axiom gcd-S≤S #-}

Internal predicates

Associate with each n -ary predicate symbol P a constant symbol $\hat{P}$, the corresponding *internal predicate*. For each arity n introduce $n + 1$ -ary relation symbols $\mathcal{P}_n$ and $\mathcal{T}_n$ such that $\mathcal{P}_n(\hat{P}, t_1, \dots, t_n)$ means that $P(t_1, \dots, t_n)$ is a well-formed proposition and $\mathcal{T}_n(\hat{P}, t_1, \dots, t_n)$ means that $P(t_1, \dots, t_n)$ is true. For example, the induction *schema* is represented as the Agda type

$$\begin{aligned} & (P : D \rightarrow \text{Set}) \rightarrow P \text{ zero} \rightarrow \\ & (\forall \{n\} \rightarrow N \ n \rightarrow P \ n \rightarrow P \ (\text{succ } n)) \rightarrow \\ & \forall \{n\} \rightarrow N \ n \rightarrow P \ n \end{aligned}$$

which is translated into an *axiom* in first order logic:

$$\begin{aligned} & \forall p. (\forall n. \mathcal{P}_1(p, n)) \rightarrow \mathcal{T}_1(p, \text{zero}) \rightarrow \\ & (\forall n. \mathcal{T}_1(\hat{N}, n) \rightarrow \mathcal{T}_1(p, n) \rightarrow \mathcal{T}_1(p, \text{succ } n)) \rightarrow \\ & \forall n. \mathcal{T}_1(\hat{N}, n) \rightarrow \mathcal{T}_1(p, n) \end{aligned}$$

Conclusion

Advantages:

- Direct reasoning about general recursive polymorphic functional programs (pure Haskell programs)
- Use off-the-shelf automatic theorem provers
- Interpretation of extensional type theory in first order theory of combinators

Disadvantage:

- First order theorem proving does not fully replace automatic type-checking in intensional type theory.

Related approaches: NuPRL! Other recent logics for functional programs: Sparkle (for Clean), PLogic (for Haskell).

Agda seems to provide a nice interface to first order theorem provers!

Website

`http://www1.eafit.edu.co/asicard/code/fotc/`